ASSIGNMENT-3

- ## PROBLEM STATEMENT:

Design a Finite State Machine (FSM) traffic light controller that manages vehicle and pedestrian signals. The system must default to Green and sequence through Yellow to Red/Walk upon a pedestrian request (P=1). The objective is to formulate the state table using two D Flip-Flops and derive the K-Map minimized excitation and output logic equations based on this sequence.

- ## OBJECTIVE:

Use Generative AI to generate a Finite State Machine (FSM) for a traffic light controller with a pedestrian request button, derive the state table and logic equations, implement using flip-flops, and verify the design through simulation.

- ## GEN AI PROMPT:

Design a Finite State Machine (FSM) for a Traffic Light Controller with a Pedestrian Request input.

Specifications:

1. Inputs:
- Reset (Active high/low synchronous/asynchronous reset to return to initial state)
- Clock signal
- Pedestrian Button (P): 0 = No pedestrian request, 1 = Pedestrian button pressed.

2. Outputs:
- Main Road Lights: Red (MR), Yellow (MY), Green (MG)
- Pedestrian Signal: Walk (PW), Don't Walk (PDW)

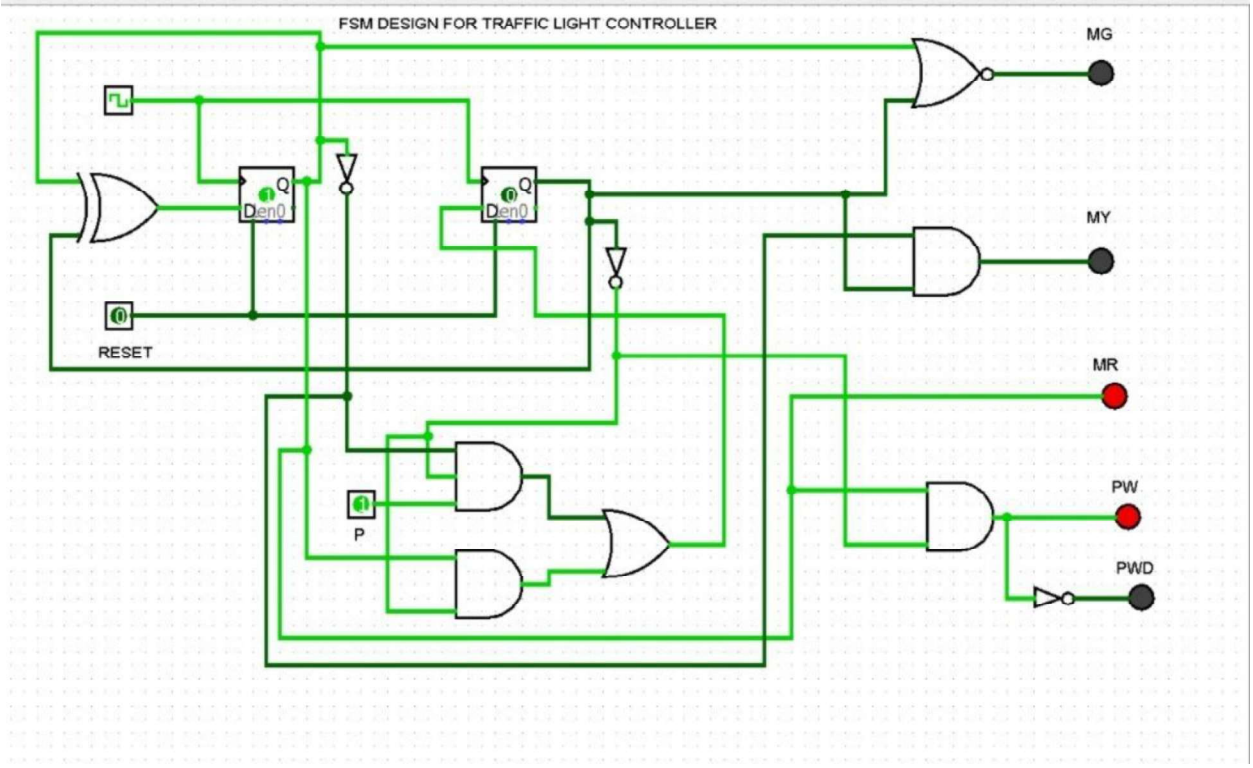
3. FSM Behavior:
- State 0 (S0): Main Green (MG=1, PW=0, PDW=1). Default state.
- If P = 1, transition to State 1 (S1): Main Yellow (MY=1, PW=0, PDW=1).
- State 2 (S2): Main Red / Pedestrian Walk (MR=1, PW=1, PDW=0).
- State 3 (S3): Main Red / Pedestrian Warning / Transition back to S0.

4. Tasks:
a) Define the State Table (Present State, Next State, Outputs based on inputs using 2 D Flip-Flops: Q1, Q0).
b) Derived excitation logic equations for D1, D0 (Inputs to Flip-Flops).
c) Derived output logic equations for MR, MY, MG, PW, PDW.
d) Formulate the simplified Boolean expressions using K-Maps.

Response: Here is the complete finite state machine (FSM) design for your traffic light controller.

This design uses a **Moore Machine** model (where outputs depend only on the current state) and assumes a clock signal governs the timing of the yellow and red lights.

- ❑ **CIRCUIT IMPLEMETATION IN LOGISIM:**



1. State Encoding

We need 3 states, requiring two D flip-flops with state variables Q_1 (MSB) and Q_0 (LSB).

- **S0 (Green):** $Q_1 Q_0 = 00$
- **S1 (Yellow):** $Q_1 Q_0 = 01$
- **S2 (Red):** $Q_1 Q_0 = 10$
- **S3 (Unused):** $Q_1 Q_0 = 11$ (Treated as Don't Care 'X' for K-map minimization)

2. State Diagram

- **S0 (Green):** Stays in Green as long as the pedestrian button is not pressed (P=0). If pressed (P=1), it transitions to Yellow.
- **S1 (Yellow):** Stays for one clock cycle, then transitions automatically to Red (Input P is a Don't Care).
- **S2 (Red):** Stays for one clock cycle to let pedestrians cross, then transitions automatically back to Green (Input P is a Don't Care).

3. State Transition Table

Current State (Q_1 Q_0)	Input (P)	Next State (Q_{1+} Q_{0+})	Green (G)	Yellow (Y)	Red (R)
00 (S0)	0	00 (S0)	1	0	0
00 (S0)	1	01 (S1)	1	0	0
01 (S1)	X	10 (S2)	0	1	0
10 (S2)	X	00 (S0)	0	0	1
11 (Unused)	X	XX	X	X	X

(Note: X = Don't Care. For states 01 and 10, the transition happens on the next clock pulse regardless of the button state).

4. Minimized Boolean Logic Equations

Since we are using D flip-flops, the next-state $Q+$ is directly equal to the D input (D=$Q+$). Using Karnaugh maps and substituting state 11 as Don't Cares, we get the following minimized equations:

Next-State Logic (D Flip-Flop Inputs):

- $D1 = Q0$ *(Grouping the 1 at Q1 Q0 =01 with the Don't Care at 11 in the K-map).*
- $D0 = Q1 \cdot Q0 \cdot P$ *(Only one condition sets D0 =1: Current state 00 and P=1. No valid Don't Care groupings are adjacent).*

Output Logic: To ensure strict safety (avoiding multiple lights turning on simultaneously if a glitch forces the circuit into the unused state 11), we decode the outputs fully without using Don't Cares for the lights:

- $G = Q1 \cdot Q0$
- $Y = Q1 \cdot Q0$
- $R = Q1 \cdot Q0$

Self-Correction Safety Check: If the circuit accidentally enters the unused state (11), $D1 = 1$ and $D0 = 0$. On the very next clock pulse, it will automatically recover into state 10 (Red), forcing a safe pedestrian state before resuming normal operation.

You can directly map these equations into Logisim using standard AND, OR, and NOT gates alongside two D flip-flops.

- **DISCUSSION :**

FSM Design for Traffic Light Controller

1. Circuit Operation: The controller acts as a 4-state synchronous machine using two D flip-flops. It remains in State 00 (Green) as long as the pedestrian button (P) is 0. When P=1, the clock triggers the safe sequence: Green → Yellow → Red → Walk → Green.
2. Simulation & Verification: In Logisim, manual clock ticks verified that the flip-flops successfully held the "Green" state until the button was pressed. Once triggered, the sequence executed in perfect chronological order, proving the AI's Next-State logic is accurate.
3. GenAI Reflection: Using AI eliminated the tedious process of drawing state tables and solving Karnaugh maps. It instantly provided the exact Boolean equations and correctly mapped the pedestrian button as a synchronous input—ensuring the lights cycle safely through Yellow and Red rather than causing an unsafe instant change.